

Introducción a los modelos de ciclos de vida

Metodologías ágiles

Los modelos de ciclo de vida tradicionales, como el ciclo de vida de cascada o evolutivo, se presentaron como soluciones para poder llevar a cabo una planificación correcta del proyecto, y se los tomaba como un aseguramiento de la calidad que resultaría si eran aplicados correctamente. Esta creencia tenía relación con el tipo de sistemas que se estaban generando, sistemas grandes, que involucraban muchos recursos, equipos de trabajo numerosos, que incluían muchos programas, y luego solían permanecer por mucho tiempo vigentes en las organizaciones.

El problema surgía cuando todo esto se debía aplicar a pequeños proyectos, a empresas más pequeñas, y surgían diversos problemas. Lo que se demoraba en tratar de definir las acciones a tener en cuenta involucraba mucho más tiempo que la programación del sistema. Se trataba de seguir con el mismo método en esos entornos cambiantes, pero los modelos rígidos, donde una modificación era muy difícil de incorporar, no se adaptaban a esas nuevas necesidades. La incorporación del modelo iterativo e incremental mostró un avance a una nueva generación de modelos de ciclo de vida.

Así, para la década del 90, comenzaron a surgir otros desarrollos de software que propusieron la incorporación de métodos ágiles basados en enfoques iterativos, centrados en generar un software funcional de manera rápida, donde el diseño y la documentación vayan acompañando ese proceso. Kent Beck fue uno de los precursores de esta búsqueda con la publicación de su libro *Extreme Programming Explained*, donde se refiere a un método ágil conocido como XP (Extreme Programming).

Algunos otros métodos ágiles que fueron apareciendo, también muy populares, son: Scrum (publicado por Kent Schwaber y Mike Beedle en 2001), Crystal Clear (publicado por Alistair Cockburn y Jim Highsmith, en 2001), Adaptive Software Development (ASD, publicado Jennifer Stapleton, en el año 1997), Feature Driven Development (FDD, desarrollada por Jeff De Luca), entre otros.

Así, para el 2001, se reunieron varios críticos de los modelos tradicionales para buscar la mejora en los procesos de desarrollo. De esta reunión surgió lo que se denominó el

Manifiesto Ágil, firmado por: Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland y Dave Thomas.

Los principios de los métodos ágiles son:

- Participación del cliente en el proceso de desarrollo.
- Desarrollo de software en forma incremental.
- Priorizar personas y no procesos, para que lleven a cabo sus propias formas de trabajo.
- Aceptar que los requerimientos del sistema cambian.
- Mantener la simplicidad en el producto y en el proceso de desarrollo.

Así, las metodologías ágiles surgieron de la experiencia de desarrollos rápidos y el uso de prototipados, tratando de alinearse más a los entornos cambiantes provocados por el avance en las tecnologías de información y comunicaciones, y la programación resurgiendo más como un arte que una ciencia.

Programación extrema (XP)

Creada por Kent Beck, Extreme Programming (XP) plantea cuatro prácticas básicas:

Liberación limitada: busca entregar rápidamente desarrollos aun cuando no estén completos porque el cliente lo requiere con urgencia, y se le incorporarán las mejoras o faltantes luego.

Semana laboral de 40h: sin agregar tiempo extra de trabajo para evitar el agotamiento de los programadores.

Cliente en el sitio: su presencia es fundamental para determinar los requerimientos del sistema. Debe estar involucrado en el proceso.

Programación en pares: dos programadores trabajan juntos para hacerlo de forma más eficiente y eficaz.

Basado en lo dicho previamente, se describe escenarios (historias de usuario) así, en lugar de definir una lista de funciones requeridas por el sistema, el cliente (como dijimos previamente debe estar muy involucrado) describe sus necesidades y determina cómo priorizarlas.

La gráfica de este modelo se visualiza a continuación:

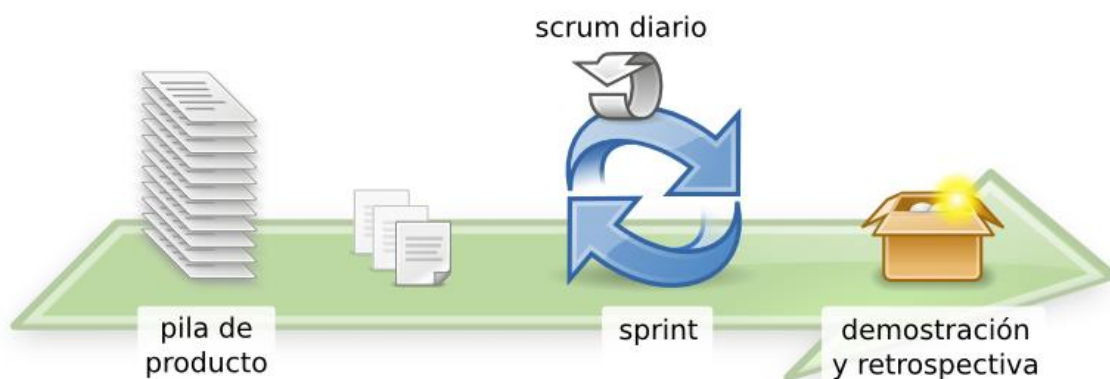


El cliente, junto al equipo de desarrollo, describen los escenarios para que estos puedan comprender las necesidades y definir las historias a implementar en la entrega. Estas historias son divididas en tareas, se planifica su entrega, se desarrolla, se prueba y se entrega el software. El cliente evaluará la entrega, y allí se vuelva a repetir el proceso, seleccionando las historias de usuario para la siguiente entrega.

Scrum

Este método propone un proceso ágil, focalizándose en agregarle valor al negocio en el menor tiempo posible, favoreciendo el desarrollo iterativo e incremental, para la mejora de proyectos y productos.

Veamos la gráfica del modelo:



El proyecto avanza a través de ciclos de trabajo denominados sprints, cuya duración varía entre una y cuatro semanas. Al iniciarse el sprint, cada equipo multidisciplinar y autogestionado seleccionará una serie de requisitos del cliente de una lista priorizada, a los que se abocará y que se compromete a concluir en el tiempo pautado para el sprint. Equipo, calidad y alcance se mantienen inalterables durante el proceso. Se centra en la gestión, no en la “ingeniería”.

En Scrum se definen tres roles:

- **Product Owner** (dueño del producto): es una única persona responsable de administrar el product Backlog y de asegurar que el equipo agregue valor a la compañía con su trabajo. Define prioridades de las funcionalidades para la lista de sprints. Sus decisiones deben ser respetadas.
- **Equipo**: construye el producto a través de los sprints. Se trata de un equipo multidisciplinario (conformado por analistas, arquitectos, programadores, diseñadores de interfaz, testers, etc.) autogestionado. No tienen líder o jefe por lo que dentro del equipo no existen títulos o jerarquías. Es deseable que los roles puedan ser intercambiados según las necesidades en cada sprints. La cantidad de integrantes de cada equipo puede ser de siete personas, más o menos dos.
- **Scrum Master**: ayuda al equipo y a la organización a usar Scrum, a que lo comprendan y lo sigan, destacando los principios de autogestión y multidisciplinaridad. No es el jefe del equipo o del proyecto, sino que es responsable de realizar coaching para lograr productividad y calidad. Trabaja para ayudar a resolver impedimentos o amenazas que pongan en riesgo la efectividad del Product Owner y el equipo. Es importante mencionar que Scrum Master y Product Owner nunca pueden ser la misma persona.

Combinación de paradigmas y selección de modelo de ciclo de vida

Si bien en este módulo (como en el anterior) hemos tratado diferentes modelos de ciclos de vida, no podemos indicar que uno sea mejor que el otro. Todo dependerá de las circunstancias y características específicas de cada proyecto. También existe la posibilidad de considerar la combinación posible entre los paradigmas vistos. Por ejemplo, si aplicamos un modelo de ciclo de vida de espiral, en la etapa o fase de desarrollo, podríamos llevar a cabo el mismo mediante la aplicación de un modelo de cascada.

Al iniciarse un nuevo proyecto, quien se encuentre a cargo tiene la responsabilidad de:

- Llevar a cabo un análisis de las circunstancias ambientales del proyecto.
- Diseñar el modelo específico de ciclo de vida para ese proyecto.
- Definir las actividades sobre el modelo.
- Desarrollar el plan para la gestión del ciclo de vida del proyecto.

En esta selección del modelo a aplicar, deben considerarse diversos aspectos a tener en cuenta, como por ejemplo:

- Posibilidad de descomposición del sistema en subsistemas de software, con agendas y entregas diferenciadas.
- Estabilidad esperada de los requisitos.
- Novedad del proceso o procesos gestionados por el sistema en el entorno del cliente.
- Criticidad de las agendas y presupuestos.
- Grado de complejidad del interfaz de operación, criticidad de la usabilidad.
- Grado de conocimiento y familiaridad con el entorno de desarrollo, componentes externos empleados, etc.
- Riesgos del proyecto.
- Visibilidad del grado de avance del proyecto para el cliente.

Una mala elección del modelo de ciclo de vida para su proyecto puede conducirlo al fracaso.